Newton School of Technology

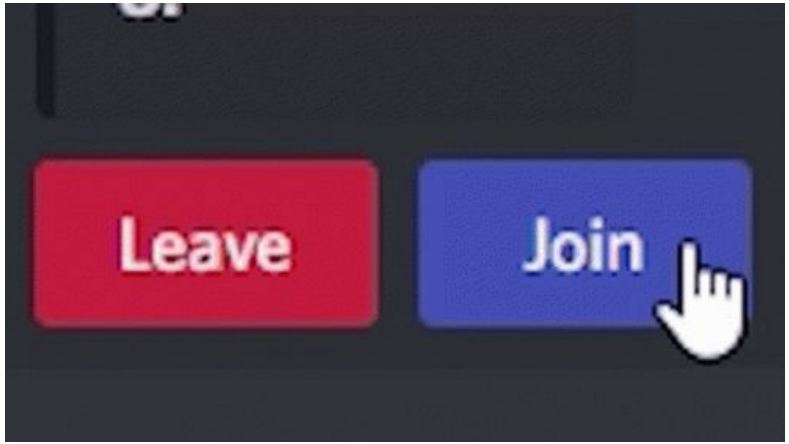
17

Rapid Prototyping Tools with Python & GenAI

Transform your ideas into working applications today.

Join the Lecture on NS Portal :)

Newton School of Technology



Quick Recap

Newton School of Technology

Kindly add here...

The Core Challenge We're Solving

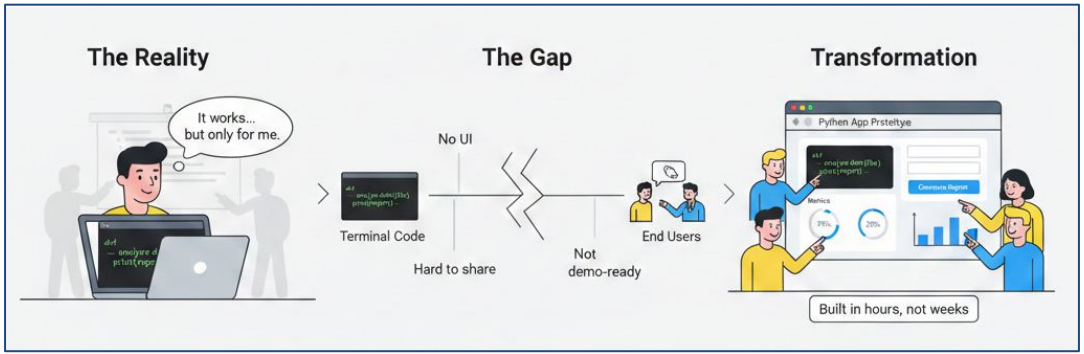
Newton School of Technology

The Problem

- Your Python code works in the terminal, but scripts aren't usable for others.
- Non-technical users need interfaces, and stakeholders expect clickable demos.

The Solution

- Learn how to turn Python logic into professional, shareable applications in hours.
- No HTML. No CSS. No JavaScript.
- Just Python and the right rapid prototyping tools.



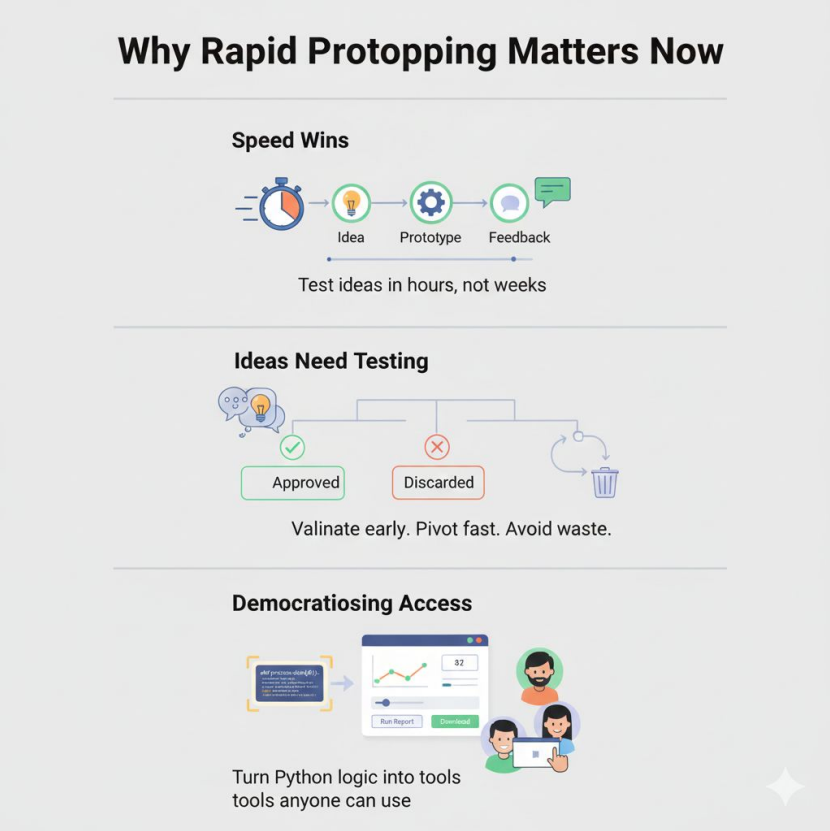
Why Rapid Prototyping Matters Now

Newton School of Technology

Speed Wins - Build and test ideas in hours, not weeks, and stay ahead of fast-moving markets.

Ideas Need Testing - Validate assumptions early, gather feedback fast, and avoid wasting effort on the wrong ideas.

Democratising Access - Turn Python logic into tools that non-technical teams can actually use.

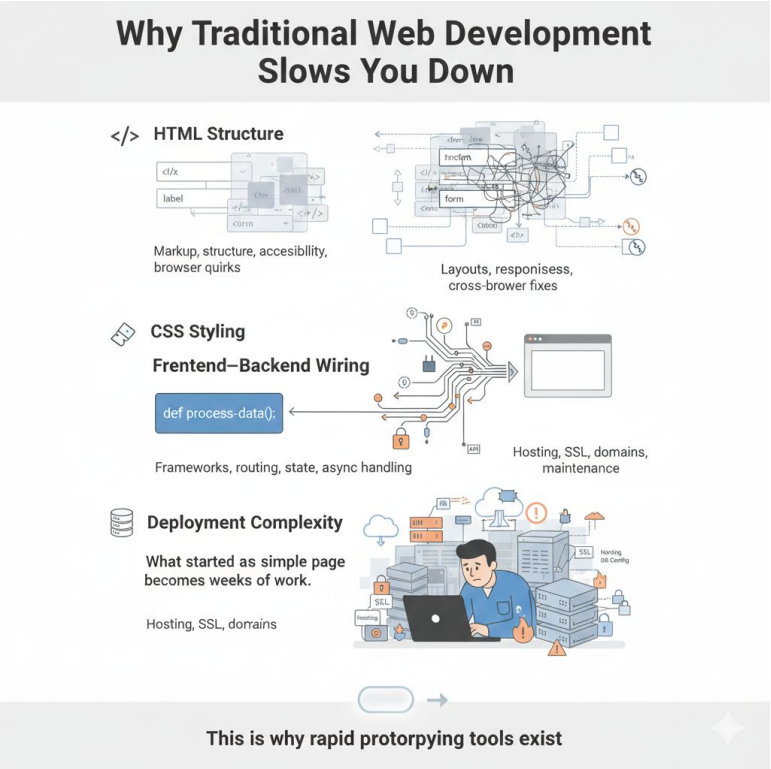


Why Traditional Web Development Slows You Down

Newton School of Technology

- **HTML Structure -** Even a simple form needs many rules, tags, and browser-specific fixes.
- **CSS Styling -** Time is spent fixing layouts and screen sizes instead of writing logic.
- **Frontend–Backend Wiring -** Connecting Python to the screen requires learning frameworks and handling requests.
- **Deployment Overhead -** You must manage servers, domains, security, and regular upkeep.

What begins as a “simple web page” often becomes weeks of work. This is exactly why rapid prototyping tools exist.



Enter Streamlit: Python-Only Web Apps

Newton School of Technology

How Streamlit Works

- If you can write Python, you can build web apps
- No HTML templates, CSS files, or JavaScript frameworks
- UI and logic live together in one Python file
- Each UI element is just a Python function call

Write Python, Get UI

- "*Import Streamlit*" and call simple functions
- Use inputs like text boxes - "`st.text_input()`" and buttons - "`st.button()`"
- See changes instantly in the browser
- No frontend setup required

Automatic Reactivity

- Code reruns automatically when users interact
- No need to manage events or callbacks
- No manual state handling for basic apps

Built-In Professional Design

- Clean and modern UI by default
- Responsive layouts without extra work
- Consistent styling across the app

Your First Streamlit Application

Newton School of Technology

Add Title and Description - Use `st.title()` and `st.write()` to create clear headers that tell users what your app does.

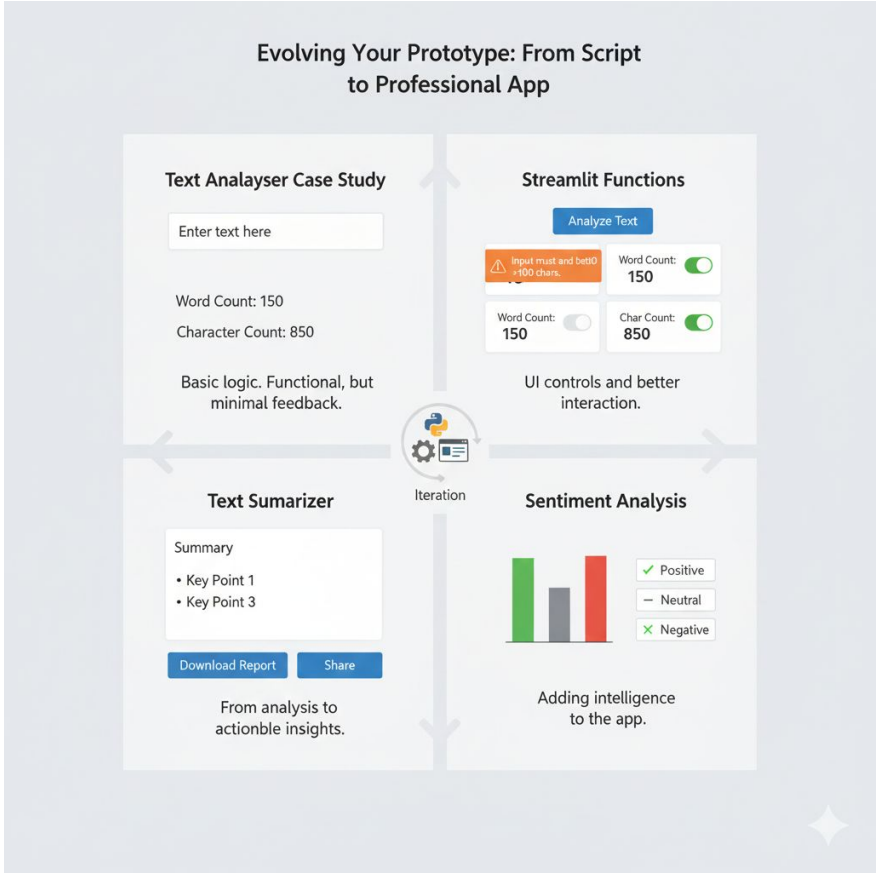
Create Text Input - Add `st.text_area()` where users can paste or type their content. This returns the entered text as a Python string.

Add Analysis Button - Include `st.button()` to trigger the analysis. This gives users control over when processing happens.

Display Results - Show the analysis using `st.metric()` for key numbers or `st.write()` for detailed output.

Evolving Your Prototype

Newton School of Technology



Evolving Your Prototype: Text Analyser Case Study

Newton School of Technology

Words & Character Counters -

*file_1.py - /Users/bipulkumar/Documents/L17/f

```python
import streamlit as st

st.title("Simple Text Analyzer")

user_text = st.text_area("Enter your text here")

if user_text:
    word_count = len(user_text.split())
    char_count = len(user_text)

    st.write("Word Count:", word_count)
    st.write("Character Count:", char_count)
```

...s/L17 — open ◂ streamlit run file_3.py ...— open ◂ streamlit run file_1.py

```
Last login: Mon Dec 29 12:36:43 on ttys000
bipulkumar@Bipuls-MacBook-Pro L17 % streamlit run file_1.py

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8502
  Network URL: http://10.6.0.192:8502

  For better performance, install the Watchdog module:

  $ xcode-select --install
  $ pip install watchdog
```

localhost:8501

Simple Text Analyzer

Enter your text here

Hi
I am learning Rapid Prototyping for AI-ML.

Word Count: 8

Character Count: 45

Evolving Your Prototype: Text Analyser Case Study

Newton School of Technology

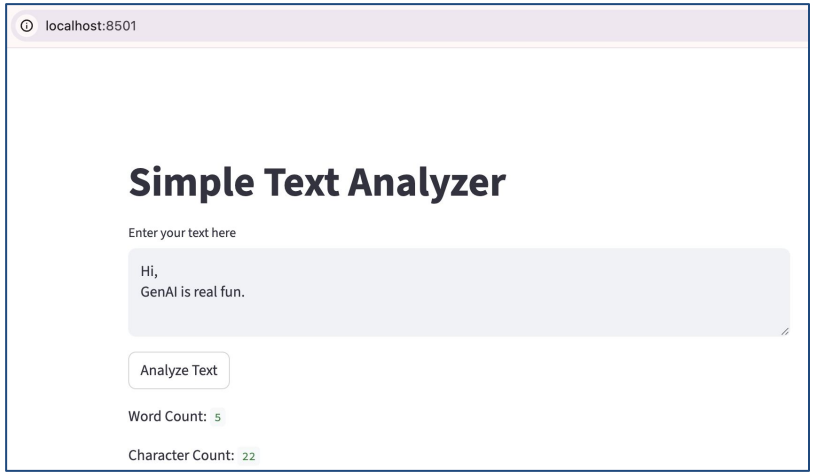
Words & Character Counters Using Button-

file_3.py - /Users/bipulkumar/Documents/L17/file_3.py (3.9.0)

```python
import streamlit as st


def analyze_text(text):
    word_count = len(text.split())
    char_count = len(text)
    return word_count, char_count


st.title("Simple Text Analyzer")

user_text = st.text_area("Enter your text here")

analyze_button = st.button("Analyze Text")

if analyze_button:
    if user_text.strip() == "":
        st.warning("Please enter some text before analyzing.")
    else:
        word_count, char_count = analyze_text(user_text)

        st.write("Word Count:", word_count)
        st.write("Character Count:", char_count)
```

L17 — open ◂ streamlit run file_3.py — 80×24

```
Last login: Mon Dec 29 12:32:13 on ttys001
bipulkumar@Bipuls-MacBook-Pro L17 % streamlit run file_3.py

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://10.6.0.192:8501

  For better performance, install the Watchdog module:

  $ xcode-select --install
  $ pip install watchdog
```

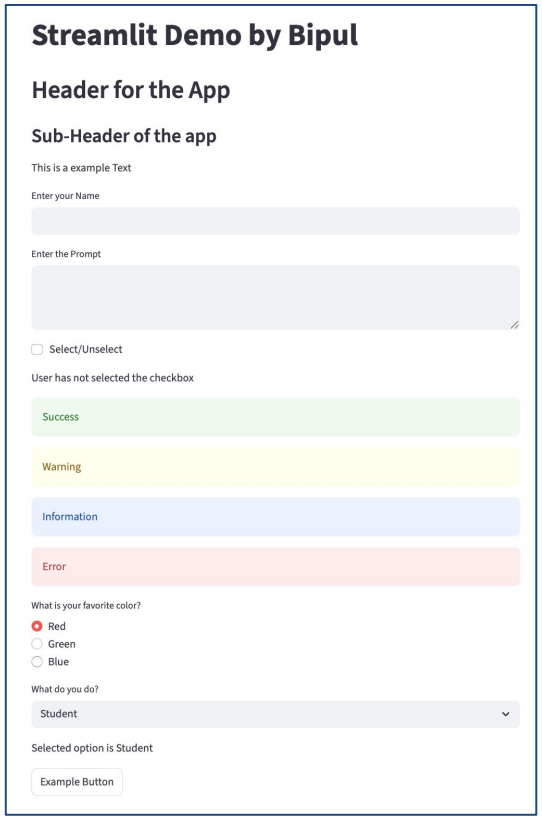


Evolving Your Prototype: Streamlit Functions

Newton School of Technology

file_9.py - /Users/bipulkumar/Documents/L17/file_

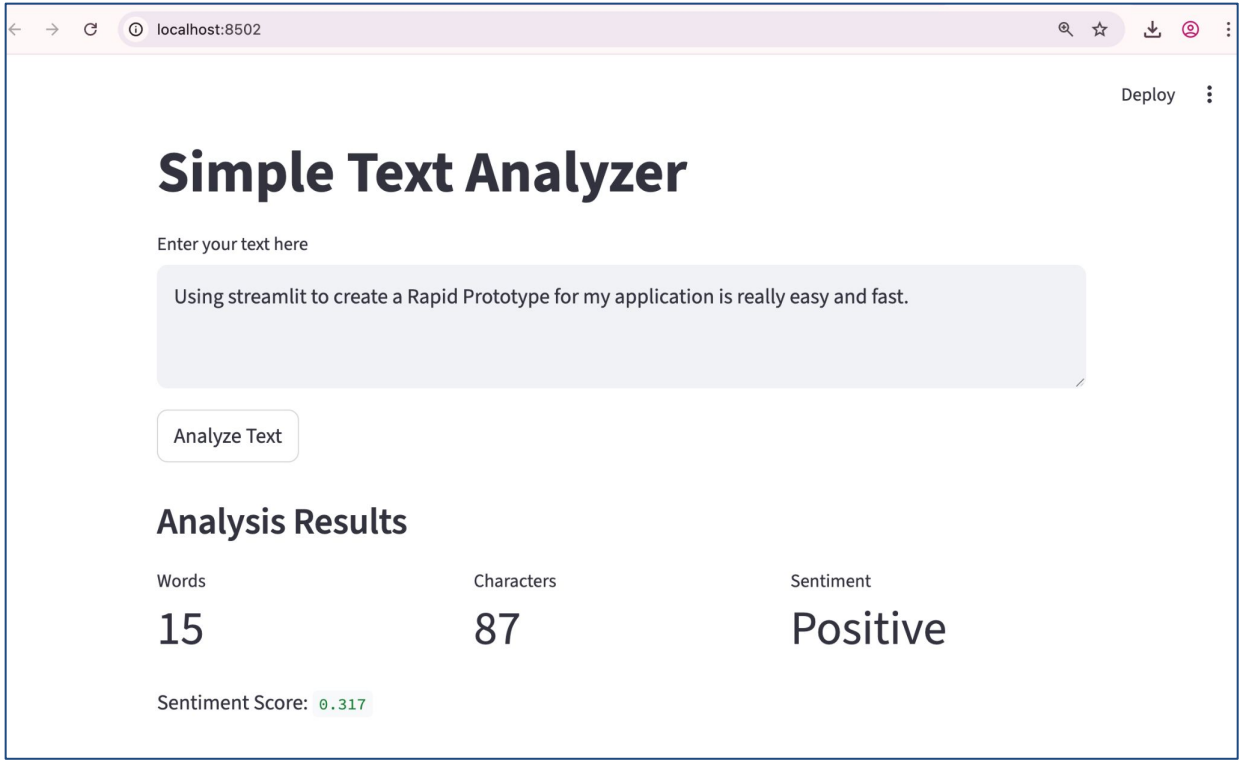
```python
import streamlit as st

st.title("Streamlit Demo by Bipul")

st.header("Header for the App")

st.subheader("Sub-Header of the app")

st.text("This is a example Text")

st.text_input("Enter your Name")
st.text_area("Enter the Prompt")

#Checkbox
#Checkbox
if st.checkbox("Select/Unselect"):
    st.text("User selected the checkbox")
else:
    st.text("User has not selected the checkbox")

st.success("Success")
st.warning("Warning")
st.info("Information")
st.error("Error")

#Radio Button
state = st.radio("What is your favorite color?" , ("Red","Green","Blue"))

if state == 'Green':
    st.success("That is my favorite color as well")

#SelectBox
occupation = st.selectbox("What do you do?" , ["Student","Vlogger","Engineer"])
st.text(f"Selected option is {occupation}")

#Button
if st.button("Example Button"):
    st.success("You clicked it")
```



Evolving Your Prototype: Sentiment Analysis

Newton School of Technology

```python
import streamlit as st
from textblob import TextBlob


def analyze_text(text):
    analysis = TextBlob(text)
    polarity = analysis.sentiment.polarity

    if polarity > 0:
        sentiment = "Positive"
    elif polarity < 0:
        sentiment = "Negative"
    else:
        sentiment = "Neutral"

    result = {
        "word_count": len(text.split()),
        "char_count": len(text),
        "sentiment": sentiment,
        "polarity_score": round(polarity, 3)
    }

    return result


st.title("Simple Text Analyzer")

user_text = st.text_area("Enter your text here")

analyze_button = st.button("Analyze Text")

if analyze_button:
    if user_text.strip() == "":
        st.warning("Please enter some text before analyzing.")
    else:
        results = analyze_text(user_text)

        st.subheader("Analysis Results")

        col1, col2, col3 = st.columns(3)

        with col1:
            st.metric("Words", results["word_count"])

        with col2:
            st.metric("Characters", results["char_count"])

        with col3:
            st.metric("Sentiment", results["sentiment"])

        st.write("Sentiment Score:", results["polarity_score"])
```



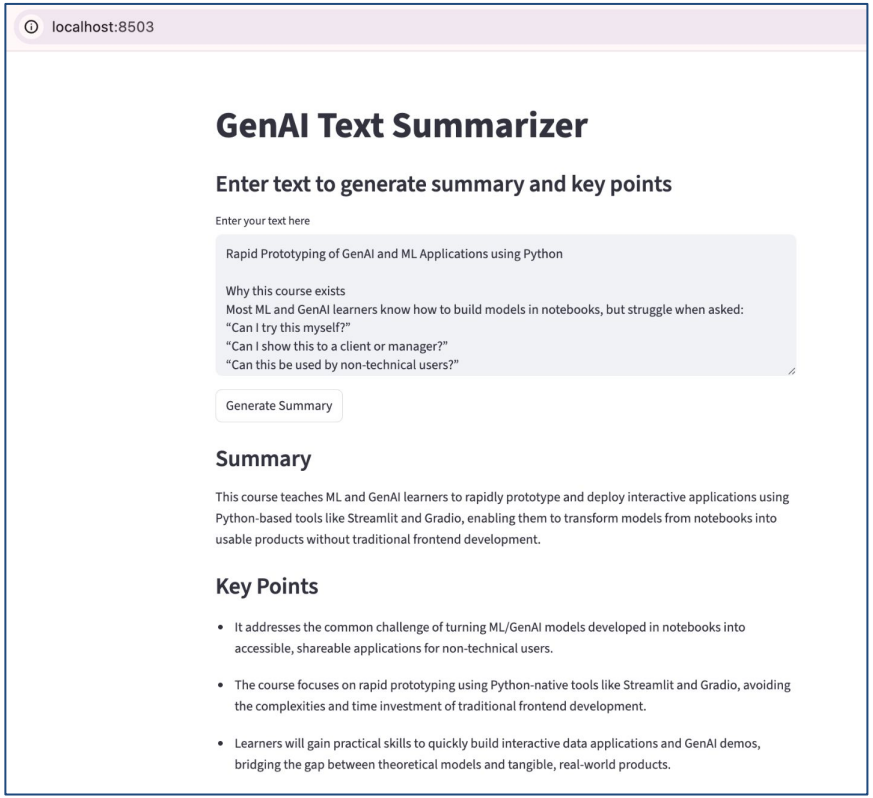
Evolving Your Prototype: Text Summarizer

Newton School of Technology

*file_8.py - /Users/bipulkumar/D

```python
import streamlit as st
from langchain_google_genai import ChatGoogleGenerativeAI
import os

# 1. SETUP: Make sure you put your actual key here
os.environ['GOOGLE_API_KEY'] = 'use gemini API Here'
# generate API key from here - https://aistudio.google.com/api-keys

st.title("GenAI Text Summarizer")
st.subheader("Enter text to generate summary and key points")

user_text = st.text_area("Enter your text here", height=200)


llm = ChatGoogleGenerativeAI(model='gemini-2.5-flash')

def generate_insights(text):
    prompt = (
        f"Analyze the following text.\n"
        f"Line 1: Provide a concise summary.\n"
        f"Line 2 onwards: Provide exactly 3 bullet points starting with a hyphen (-).\n"
        f"Text to analyze:\n{text}"
    )


    response = llm.invoke(prompt)


    lines = response.content.split('\n')


    clean_lines = [line.strip() for line in lines if line.strip()]

    summary = clean_lines[0] if clean_lines else "No summary generated."
    key_points = clean_lines[1:] if len(clean_lines) > 1 else []
    #summary = str(clean_lines)


    return {
        "summary": summary,
        "key_points": key_points
    }

if st.button("Generate Summary") and user_text.strip():
    with st.spinner("Generating insights..."): # Added a loading spinner for better UX
        try:
            results = generate_insights(user_text)

            st.subheader("Summary")
            st.write(results["summary"])

            st.subheader("Key Points")
            for point in results["key_points"]:
                st.write(point)
        except Exception as e:
            st.error(f"An error occurred: {e}")
```

App Sto



Gradio: Function-First Interfaces

Newton School of Technology

Why Gradio After Streamlit?

- Streamlit is great for full data applications
- But sometimes all you need is to test or demo a single function
- Ideal for quick testing and stakeholder demos
- This is where Gradio fits naturally

Different Philosophy

- Streamlit builds complete applications
- Turn any Python function into a web UI
- You define inputs, outputs, and logic
- Gradio creates the interface automatically

ML-Native Design

- Requires very little code
- Built specifically for machine learning demos
- Handles images, text, numbers, and predictions easily
- Works well with classification, confidence scores, and outputs

Gradio: Function-First Interfaces

Newton School of Technology

Why Gradio After Streamlit?

- Streamlit is great for full data applications
- But sometimes all you need is to test or demo a single function
- Ideal for quick testing and stakeholder demos
- This is where Gradio fits naturally

Use Streamlit When

- Building data exploration tools
- Creating dashboards with many components
- Needing custom layouts and workflows
- Developing internal business apps
- Working with multi-page applications

Use Gradio When

- Demoing machine learning models
- Wrapping a single function quickly
- Testing model inputs and outputs
- Sharing lightweight ML demos
- Keeping UI code minimal

Building Your First Gradio Interface

Newton School of Technology

Let's start with something simple: a greeting function. This demonstrates Gradio's core concept without complexity.

Step 1: Start with a Simple Function

- Begin with a basic greeting function
- It takes a name as input
- It returns a greeting message
- This is just normal Python code

Step 2: Tell Gradio About Inputs and Outputs

- Tell Gradio what input your function needs
- In this case, a text box for the name
- Tell Gradio what the function returns
- Gradio creates the UI for you

Step 3: Run the App

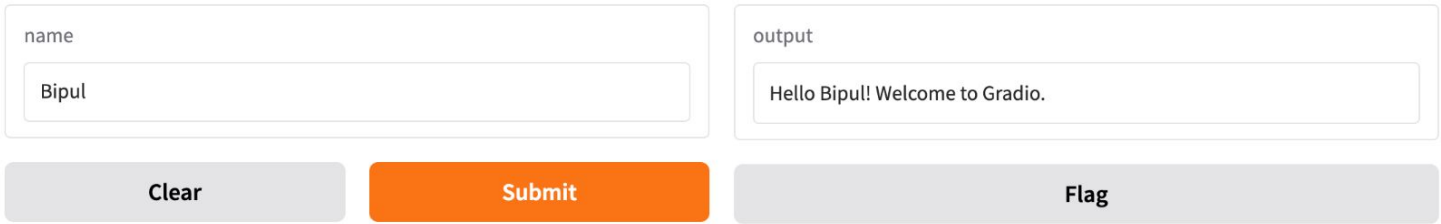
- Call the .launch() method
- Gradio starts a local web app
- Your browser opens automatically
- You can interact with your function on the screen

Building Your First Gradio Interface

Newton School of Technology

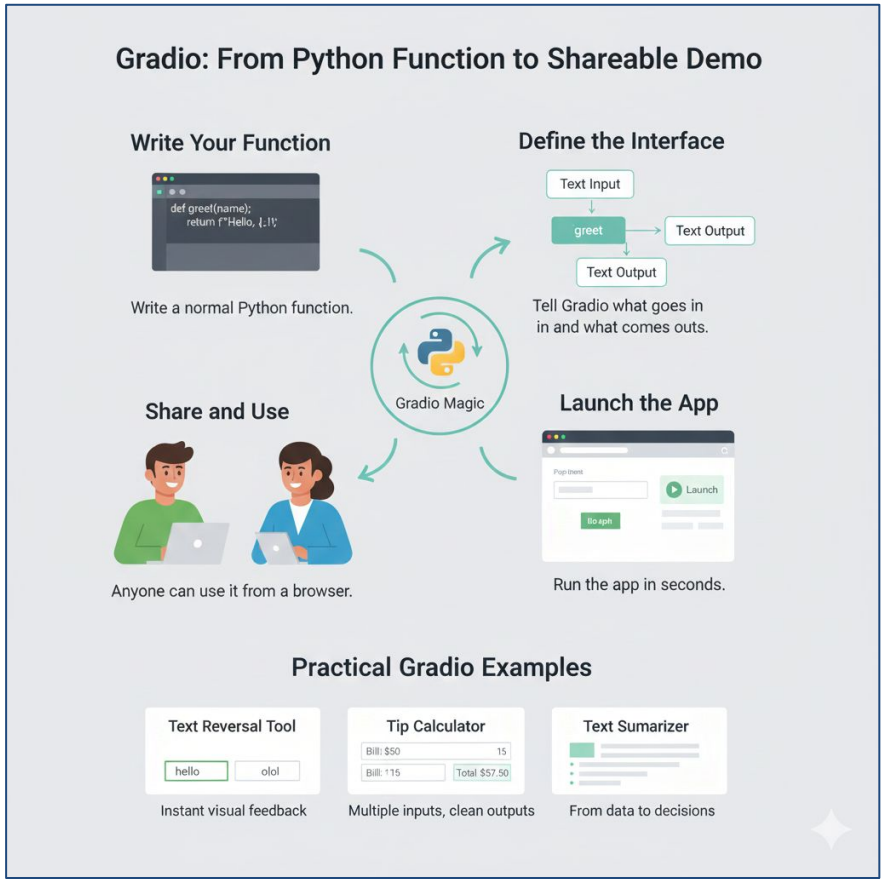
```python
import gradio as gr

# 1. The Logic: A simple function
def greet(name):
    return f"Hello {name}! Welcome to Gradio."

# 2. The Interface: Connecting the function to the UI
demo = gr.Interface(fn=greet, inputs="text", outputs="text")

# 3. The Launch: Starting the server
demo.launch()
```

```
It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`. Automatically setting `share=True` (you can

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://f7139b3f15184a5385.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working directo
```



Practical Gradio Examples

Newton School of Technology



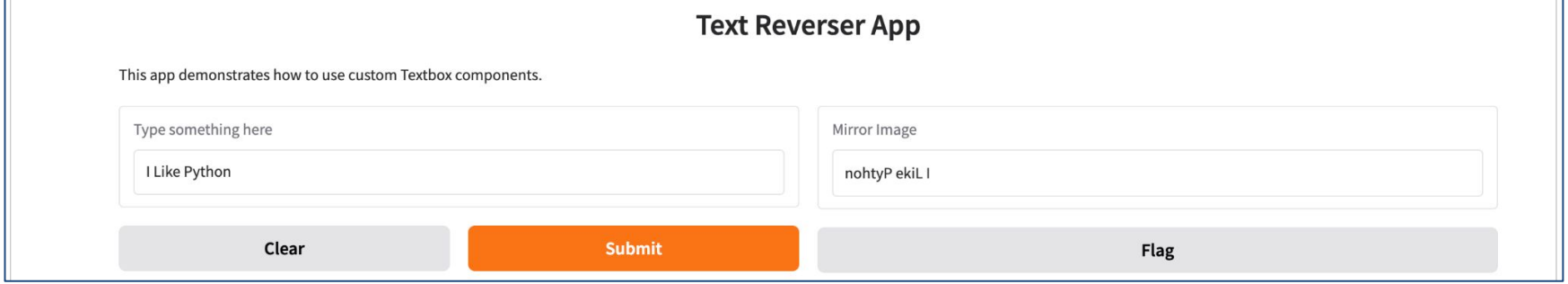
Code File - https://colab.research.google.com/drive/1lau-IvtnKRNXgulrtEugIrvkX_4MfKNH?usp=sharing

Practical Gradio Examples - Text Reversal Tool

Newton School of Technology

```python
import gradio as gr

def reverse_text(text):
    return text[::-1]

demo = gr.Interface(
    fn=reverse_text,
    inputs=gr.Textbox(label="Type something here", placeholder="e.g. Gradio is cool"),
    outputs=gr.Textbox(label="Mirror Image"),
    title="Text Reverser App",
    description="This app demonstrates how to use custom Textbox components."
)

demo.launch()
```

```
It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`. Automatically setting `share=True` (you can turn this off by

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://b9bf021d4e956d3cd8.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working directory to deploy to
```

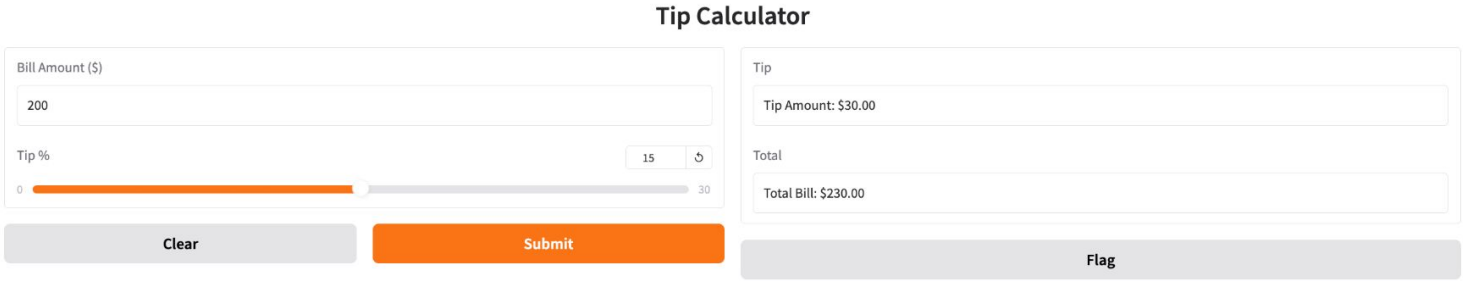


Practical Gradio Examples – Tip Calculator

Newton School of Technology

```python
import gradio as gr

def calculate_tip(bill_amount, tip_percentage):
    tip = bill_amount * (tip_percentage / 100)
    total = bill_amount + tip
    return f"Tip Amount: ${tip:.2f}", f"Total Bill: ${total:.2f}"

demo = gr.Interface(
    fn=calculate_tip,
    # Pass a LIST of inputs
    inputs=[
        gr.Number(label="Bill Amount ($)"),
        gr.Slider(minimum=0, maximum=30, value=15, label="Tip %")
    ],
    # Pass a LIST of outputs
    outputs=[
        gr.Textbox(label="Tip"),
        gr.Textbox(label="Total")
    ],
    title="Tip Calculator"
)

demo.launch()
```

```
It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`. Automatically setting `share=True` (you can turn this off by setting `share=False` in `launch()` explicit

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://82f720afb003010dc6.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working directory to deploy to Hugging Face Spaces (https://huggingface.co/s
```



Practical Gradio Examples – Tip Calculator

Newton School of Technology

```python
import gradio as gr
from langchain_google_genai import ChatGoogleGenerativeAI
import os

# 1. SETUP: Make sure you put your actual key here
os.environ['GOOGLE_API_KEY'] = 'Use Gemini API Here'
# generate API key from here - https://aistudio.google.com/api-keys

# Initialize LLM (Ensure you use a valid model name like 'gemini-1.5-flash-001' or 'gemini-pro')
llm = ChatGoogleGenerativeAI(model='gemini-2.5-flash')

def generate_insights(text):
    # Handle empty input
    if not text or not text.strip():
        return "Please enter some text.", ""

    prompt = (
        f"Analyze the following text.\n"
        f"Line 1: Provide a concise summary.\n"
        f"Line 2 onwards: Provide exactly 3 bullet points starting with a hyphen (-).\n"
        f"Text to analyze:\n{text}"
    )

    try:
        response = llm.invoke(prompt)

        # Parse response
        lines = response.content.split('\n')
        clean_lines = [line.strip() for line in lines if line.strip()]

        summary = clean_lines[0] if clean_lines else "No summary generated."

        # Join the list of points into a single string for the text box
        key_points_list = clean_lines[1:] if len(clean_lines) > 1 else []
        key_points_str = "\n".join(key_points_list)

        # Return two values: one for the Summary box, one for the Key Points box
        return summary, key_points_str

    except Exception as e:
        return f"An error occurred: {e}", ""

# 2. GRADIO UI SETUP
app = gr.Interface(
    fn=generate_insights,  # The function to call
    inputs=gr.Textbox(lines=10, placeholder="Enter your text here...", label="Input Text"),
    outputs=[
        gr.Textbox(label="Summary", lines=2),
        gr.Textbox(label="Key Points", lines=5)
    ],
    title="GenAI Text Summarizer",
    description="Enter text to generate a summary and key points using Google Gemini."
)

# 3. LAUNCH
if __name__ == "__main__":
    app.launch()
```

```
It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`. Automatically setting `share=True` (you can turn this off by setting `share=False` in `launch()` explicitly).

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://47416766e54922ad99.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working directory to deploy to Hugging Face Spaces (https://huggingface.co/spaces)
```

GenAI Text Summarizer

Enter text to generate a summary and key points using Google Gemini.

Input Text

Rapid Prototyping of GenAI and ML Applications using Python

Why this course exists
Most ML and GenAI learners know how to build models in notebooks, but struggle when asked:
"Can I try this myself?"
"Can I show this to a client or manager?"
"Can this be used by non-technical users?"
This course focuses on turning ML and GenAI ideas into usable applications quickly, without getting stuck in traditional frontend development.

Who this course is for
Beginners in Machine Learning and Data Science
Python developers curious about GenAI applications
ML engineers who want to showcase models as real products
Students who want practical, industry-style learning
No prior UI or web development experience is expected.

How the course progresses
The course moves in a very natural way:

Summary

This course teaches rapid prototyping of GenAI and ML applications using Python, enabling learners to quickly transform models into usable, interactive tools without traditional frontend development.

Key Points

- The course addresses the common challenge ML/GenAI learners face in turning models built in notebooks into shareable, client-ready applications.
- It achieves this by focusing on Python-based rapid prototyping tools like Streamlit and Gradio, which allow developers to build UIs quickly without delving into traditional web development.
- Learners will gain practical skills to bridge the gap between ML models and user interaction, creating tangible applications for diverse GenAI and ML use cases.

Next steps: Deploy to Cloud Run

What You Can Build Now

Newton School of Technology

Internal Data Tools - Transform CSV files into interactive explorers. Let colleagues filter, visualise, and analyse data without touching code or spreadsheet formulas.

Text Analysis Apps - Build sentiment analysers, readability checkers, or keyword extractors. Empower content teams with instant feedback on their writing.

Machine Learning Demos - Showcase trained models to stakeholders. Create testing interfaces that let non-technical users validate predictions and understand model behaviour.

GenAI Summarisers - Deploy document summarisers, report generators, or content analysers. Let your team leverage AI capabilities through simple, focused interfaces.

Shareable Prototypes - Turn rough ideas into clickable demos within hours. Gather feedback, validate concepts, and iterate rapidly before committing to full development.

Business Calculators - Create pricing tools, ROI calculators, or scenario planners. Replace static spreadsheets with interactive apps that provide immediate insights.

Your Rapid Prototyping Journey

Newton School of Technology

Skills We've Gained

- Building web UIs using pure Python
- Creating Streamlit data applications
- Wrapping functions with Gradio
- Integrating pre-built NLP libraries
- Adding GenAI capabilities to prototypes
- Choosing the right tool for each problem
- Iterating designs based on user needs
- Deploying shareable demos quickly

Thank You !!

See You Guys in Next Session :)